7) Write a C program to simulate page replacement algorithm

a) FIFO b) LRU (least recently used) c) Optimal.

```
#include <stdio.h>
#define MAX 50
int isPresent (int frames[], int n, int page) // fn to check if page is present
{   for (int i = 0; i < n; i++) {
        if (frames[i] == page)
            return 1;
    }
    return 0;
}
void FIFO (int pages[], int n, int capacity) {
    int frames[MAX], index = 0, faults = 0;
    for (int i = 0; i < capacity; i++) {
        frames[i] = -1;
    printf ("\n\n FIFO Page Replacement :\n");
    for (int i = 0; i < n; i++) {
        if (!isPresent (frames, capacity, pages[i])) {
            frames[index] = pages[i];
            index = (index + 1) % capacity;
            faults++;
        }
        printf ("Page %d -> ", pages[i]);
        for (int j = 0; j < capacity; j++) {
            if (frames[j] != -1)
                printf ("%d ", frames[j]);
            else
                printf ("- ");
        }
        printf ("Total page Faults = %d \n", faults);
}
```

```
void LRU (int pages[], int n, int capacity) {
    int frames[MAX], recent[MAX], faults = 0;
    for (int i=0; i<capacity; i++) {
        frames[i] = -1;
        recent[i] = -1;
    }
    printf("\n LRU Page Replacement :\n");
    for (int i=0; i<n; i++) {
        int found = 0;
        for (int j=0; j<capacity; j++) {
            if (frames[j] == pages[i]) {
                found = 1;
                recent[j] = i;
                break;
            }
        }
        if (!found) {
            int empty/lru = 0;
            for (int j=0; j<capacity; j++) {
                if (recent[j] == -1 / recent[lru]) {
                    empty/lru = j;
                    break;
                }
            }
            frames[lru] = pages[i]; if (empty != 1) {
                recent[lru] = i;
                faults++;
            } else {
                frames[empty] = pages[i];
                recent[empty] = i;
            }
        }
        printf("Page %d -> ", pages[i]);
        for (int j=0; j<capacity; j++) {
            if (frames[j] != -1) {
                int lru = 0;
                if (recent[j] <= recent[lru])
                    printf("%d ", frames[j]); lru = j;
            }
        }
        frames[lru] = pages[i];
        recent[lru] = i;
        printf("- ");
    }
    faults++;
    printf("Total Page Faults = %d\n", faults);
}
```

```
void optimal(int pages[], int n, int capacity){
    int frames[MAX], faults = 0;
    for(int i=0; i<capacity; i++){
        frames[i] = -1;
    printf("\n Optimal Page Replacement: \n");
    for(int i=0; i<n; i++){
        if(!isPresent(frames, capacity, pages[i])){
            int replace = -1, farthest = i+1;
            for(int j=0; j<capacity; j++){
                int k;
                for(k=i+1; k<n; k++){
                    if(frames[j]==pages[k]){
                        if(k>farthest){
                            farthest=k;
                            replace=j;
                        }
                        break;
                    }
                }
                if(k==n){
                    replace=j;
                    break;
                }
            }
            if(replace==-1){
                replace=0;
            frames[replace]=pages[i];
            faults++;
        }
        printf("Page %d -> ", pages[i]);
        for(int j=0; j<capacity; j++){
            if(frames[j]!=-1){
                printf("%d ", frames[j]);
            else
                printf("- ");
        }
        printf("\n");
    }
    printf("Total Page faults=%d\n", faults);
}
```

```
int main() {
    int pages[MAX], n, capacity;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter page reference string:\n");
    for(int i=0; i<n; i++) {
        scanf("%d", &pages[i]);
    }
    printf("Enter number of frames: ");
    scanf("%d", &capacity);
    FIFO(pages, n, capacity);
    LRU(pages, n, capacity);
    Optimal(pages, n, capacity);
    return 0;
}
```

O/P: Enter number of pages: 20

Enter Page reference string:

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

FIFO Page Replacement:

Page 7 -> 7 - -

Page 0 -> 7 0 -

Page 1 -> 7 0 1

Page 2 -> 2 0 1

Page 0 -> 2 0 1

Page 3 -> 2 3 1

Page 0 -> 2 3 0

Page 4 -> 4 3 0

Page 2 -> 4 2 0

Page 3 -> 4 2 3

Page 0 -> 0 2 3

Page 3 -> 0 2 3

Page 2 -> 0 2 3

Page 1 -> 0 1 3

Page 2 -> 0 1 2

Page 0 -> 0 1 2

Page 1 -> 0 1 2

Page 7 -> 7 1 2

Page 0 -> 7 0 2

Page 1 -> 7 0 1

Total Page Faults = 15

LRU Page Replacement:

Page 7 -> 7 _ _

0 -> 7 0 _

1 -> 7 0 1

2 -> 2 0 1

0 -> 2 0 1

3 -> 2 0 3

0 -> 2 0 3

4 -> 4 0 3

2 -> 4 0 2

3 -> 4 3 2

0 -> 0 3 2

3 -> 0 3 2

2 -> 0 3 2

1 -> 1 3 2

2 -> 1 3 2

0 -> 1 0 2

1 -> 1 0 2

7 -> 1 0 7

0 -> 1 0 7

1 -> 1 0 7

Total Page faults = 12

Optimal Page Replacement:

Page 7 -> 7 - -

0 -> 7 0 -

1 -> 7 0 1

2 -> 2 0 1

0 -> 2 0 1

3 -> 2 0 3

0 -> 2 0 3

4 -> 2 4 3

2 -> 2 4 3

3 -> 2 4 3

0 -> 2 0 3

3 -> 2 0 3

2 -> 2 0 3

1 -> 2 0 1

2 -> 2 0 1

0 -> 2 0 1

1 -> 2 0 1

7 -> 7 0 1

0 -> 7 0 1

1 -> 7 0 1